



Programming Mobile Applications in Flutter

Intro lecture

THIRTY SECONDS ON WHO IS TALKING

We build Flutter apps for a living. The tools we needed along the way, we open-source.

80+

experts on board

100+

products delivered for enterprises
and scale-ups

THE ONE YOU MAY ALREADY USE

Patrol

Flutter-first UI testing with native automation.

1.4k ★ · ~475k downloads a month

TRUSTED BY



LEANCODE.CO ON PUB.DEV · SIZED BY DOWNLOADS IN THE LAST 30 DAYS

48 packages, over 2 million downloads a month

dashed_line leancode_analytics_posthog lens_base inlay_gen inlay leancode_flutter_svg_adaptive_loader flutter_spyglass
login_client_flutter comms enhanced_gradients leancode_analytics_base logging_bugfender
leancode_debug_page cqrs leancode_contracts_generator arb_translate
leancode_cubit_utils webp marionette_logging leancode_lint
patrol_mcp patrol_cli **marionette_flutter** patrol_finders
patrol_log **patrol** adb **marionette_mcp**
bloc_presentation_test bloc_presentation marionette_logger
advanced_forms marionette_cli leancode_hooks leancode_contracts ciach
leancode_markup leancode_cubit_utils_cqrs leancode_analytics_firebase login_client
flutter_comms bloc_lens bloc_lens_macros override_api_endpoint inlay_compose spyglass leancode_cyberware_contract_base
address



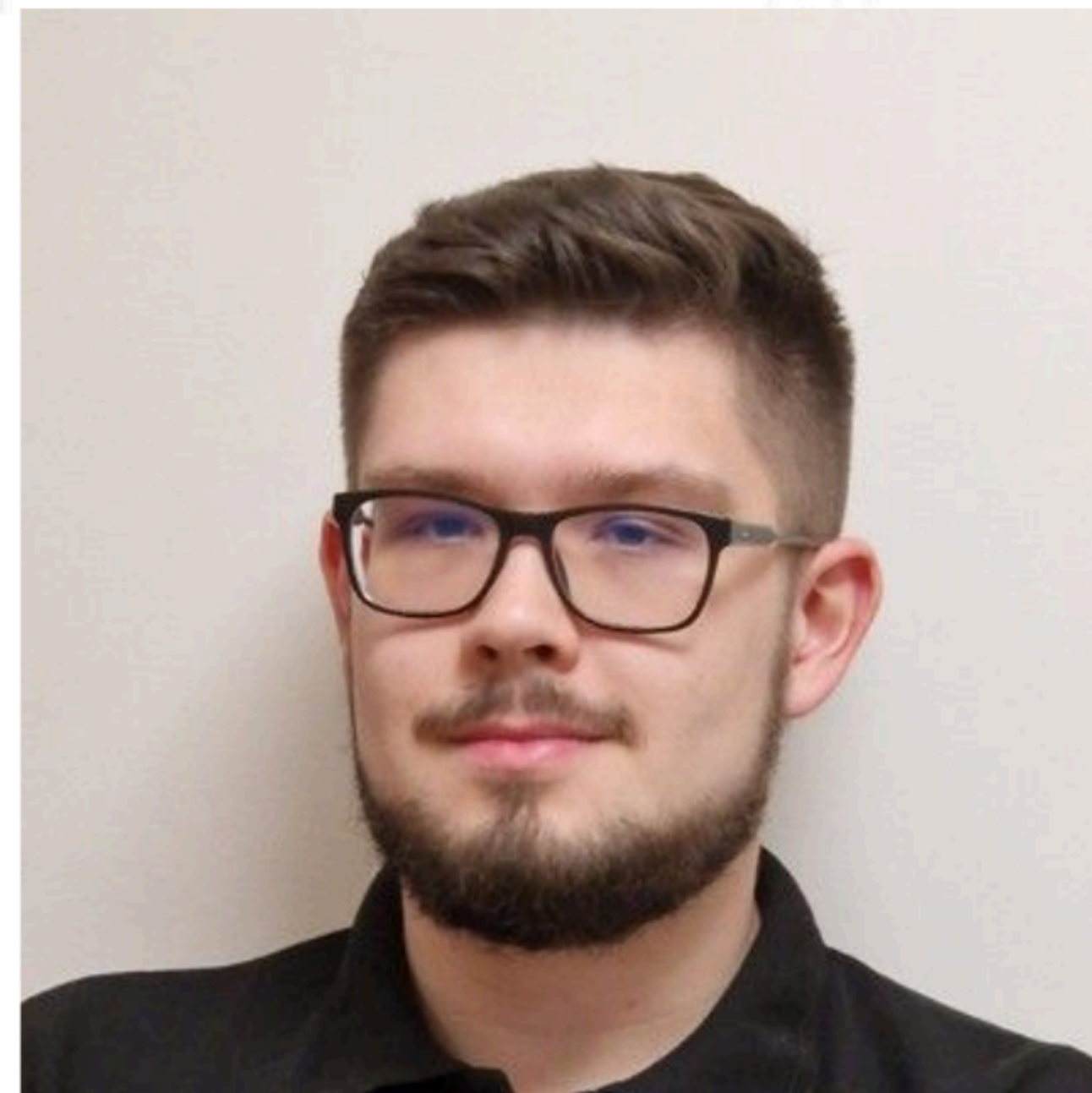
Who are we?

- inż. Mateusz Wojtczak
- Contact: mateusz.wojtczak@leancode.pl
- Head of Mobile @ LeanCode
- Flutter, React Native, Xamarin, Native
- Google Developer Expert
- producing music after work



Who are we?

- inż. Piotr Rogulski
- Contact: piotr.rogulski@leancode.pl
- Flutter Developer @ LeanCode
- Fluttercon Berlin 2024 speaker
- Author of [ciach](#) package



Who are we?

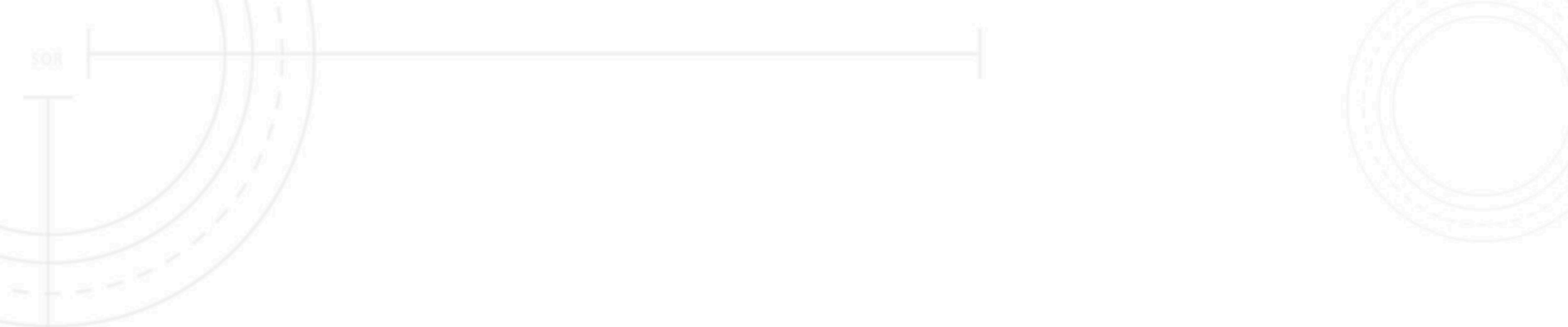
- Wiktor Zająć
- Contact: wiktor.zajac@leancode.pl
- Flutter Developer @ LeanCode



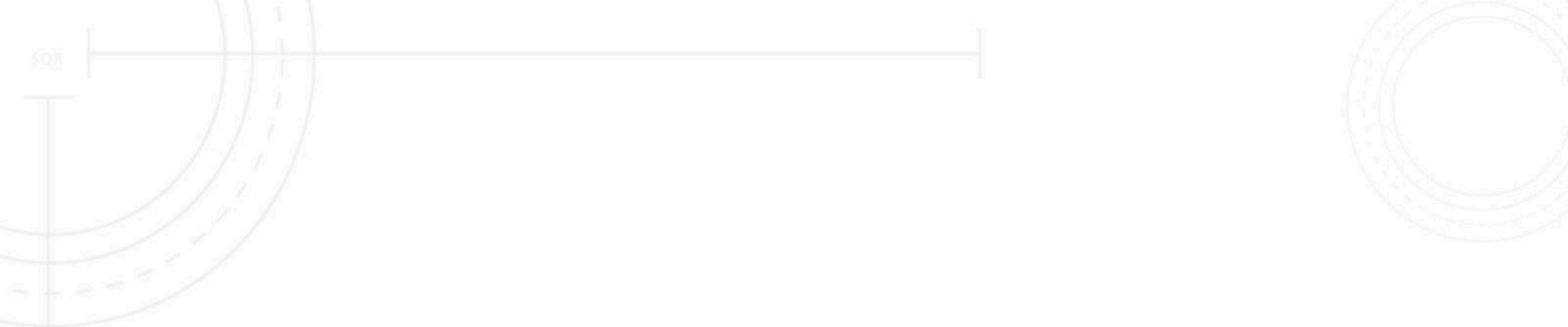
Who are we?

- inż. Kamil Sztandur
- Contact:
 - kamil.sztandur@leancode.pl
 - [Kamil Sztandur](#) (LinkedIn)
 - [@ksztandur](#) (X)
- Flutter Developer & DevRel @ LeanCode
- Tech conference speaker
- Author of [advanced_forms](#) package
- After work: gamer, coffee maker, gym enjoyer

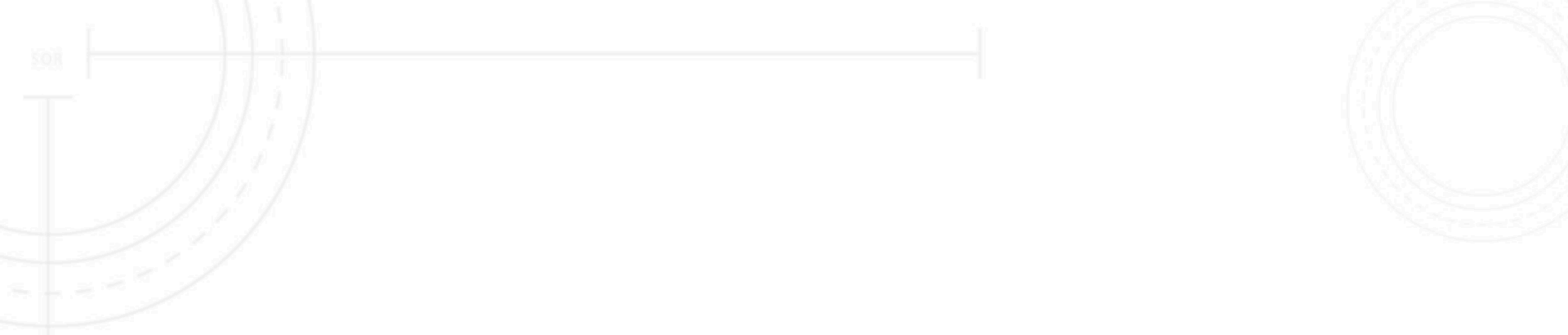




Who are you?



What is it all about?



[https://github.com/leancodepl/
flutter-at-mini](https://github.com/leancodepl/flutter-at-mini)

Rules

- Points 0-100:
 - 51-60pt - 3
 - 61-70pt - 3.5
 - 71-80pt - 4
 - 81-90pt - 4.5
 - 91-100pt - 5
- Project - 100pt
- Activity during lectures - 10pt
- Labs (non-obligatory) - $8 * 3 = 24$ pt

Lectures

1.	06.10	Introduction to Flutter	Wiktor Zając
2.	13.10	Layouts 2: Flex, lists and slivers	Wiktor Zając
3.	20.10	Giving context to BuildContext	Tomasz Koter
4.	27.10	Asynchrony and HTTP	Piotr Rogulski
5.	03.11	State Management with External Libraries	Wiktor Zając
6.	10.11	Firebase	Wiktor Zając
7.	17.11	Animations	Wiktor Zając
8.	24.11	Architecture	Mateusz Wojtczak

Lectures

9.	01.12	Testing	Wiktor Zając
10.	08.12	Forms	Kamil Sztandur
11.	15.12	Data Persistence	Piotr Rogulski
12.	22.12	Fullstack Development with .NET and Flutter	Wiktor Zając
13.	12.01	Communication with Native	Wiktor Zając
14.	19.01	Flame	Kamil Sztandur
15.	26.01	Design Systems & Accessibility	Kamil Sztandur

Labs

1. 06.10 Getting started: Dart and Flutter setup
2. 13.10 Layouts 1
3. 20.10 Layouts 2 & using context
4. 27.10 Context and StatefulWidget
5. 03.11 Communication with API
6. 10.11 State management with external services
7. 17.11 Firebase Auth in action
8. 24.11 Animations
9. 01.12 ***Mandatory*** project checkpoint

Weeks 10–15 → project consulting, by prior arrangement

Project

- Individual multi-layer Flutter application that works at least on one mobile platform (Android/iOS)
- Project topic and scope is defined by the student, should be described in the initial documentation and approved by your lab teacher

Sample projects from past editions

- Collaborative live whiteboard
- Shipment & delivery app
- “Ships” game
- “4 in a row” game
- Resume PDF generator app

Documentation

- Initial Documentation
 - Project Description
 - Use cases
- Final Documentation
 - Project description
 - Integrations
 - List of optional requirements
 - Instruction
 - Test account (if applicable)
 - Database/Firestore schema (if applicable)
 - CI/CD description/screenshot (if applicable)

Example initial docs

- Chat with Authorization
- Description
 - Screens list and short description
 - Login Screen
 - Channel List Screen
 - Message List Screen
 - ...
- Use cases
 - As a User, I can sign in using Google/Facebook/Instagram/Apple account
 - As a User, I can see a list of channels
 - As a User, I can create/delete/leave channel if I have sufficient permissions
 - As a User, I can send plain text messages
 - As a User, I can send images/videos/files
 - As a User, I can edit messages

Assessment Rules

Implementation of the required project assumptions (50pt):

- Initial documentation - 5pt
- Architecture - 15pt
- Code quality (e.g. static code analysis, formatting) - 15pt
- UI/UX
 - Material Design - 5pt
 - Custom design widgets - 5pt
- Final documentation - 5pt

Assessment Rules

Optional requirements (**max 50pt**)

- Animations
 - Implicit / ready-to-use packages - max 5pt
 - Custom - max 10pt
- Tests
 - Unit tests - max 5pt
 - Widget tests - max 5pt
 - Patrol tests - max 10pt
- Signing in process
 - Firebase Auth - max 5pt
 - Custom-backend auth - max 10pt
- Multi-step form with validation - max 10pt

Assessment Rules

Optional requirements (**max 50pt**)

- CI/CD - max 15pt
 - code analysis & run flutter test - max 10pt
 - deploy app - max 10pt
- Platform Channels
 - Using pub package for platform features (e.g. camera) - 5pt
 - Creating custom platform channels - 15pt
- Internationalization - max 10pt
- Custom painting - max 5pt
- Local data persistence (offline) - max 15pt

Timeline

- **27.10.2026** - Initial documentation (Lab 4)
- **01.12.2026** - ***Mandatory*** project checkpoint (Lab 9)
- **29.01.2027** - Project Submission
 - Source Code and Final Documentation
- **14.02.2027** - Late Project Submission
 - Each day of being late will take a decrease of **5pt** from the total number of gained points

AI

- AI tools are allowed in labs and in the project
- At the checkpoint and the defense you have to explain your code



Mitchell Hashimoto  

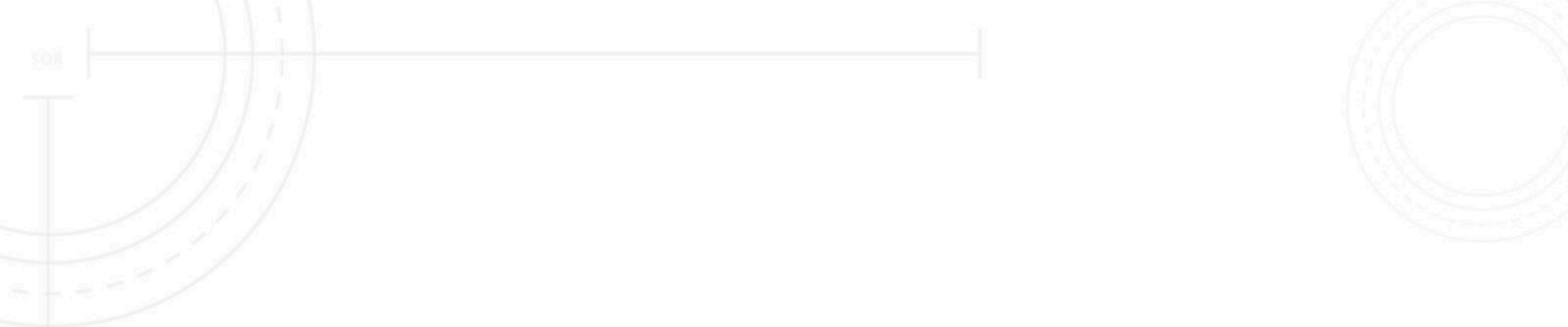
@mitchellh · [Follow](#)



The "whiteboard defense:" I should be able to pull you aside at any moment and ask you to explain any customer-facing system you've shipped. You should be able to clearly explain how it works and defend the decisions you made. This is my benchmark for responsible AI usage.

5:44 PM · Sep 16, 2026

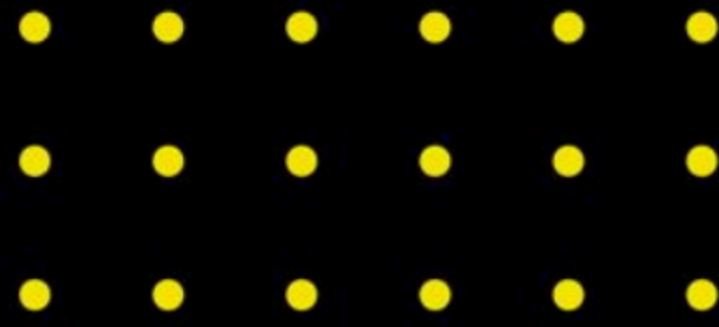




Any questions?

What is Flutter?

**Design systems are
platform-independent**

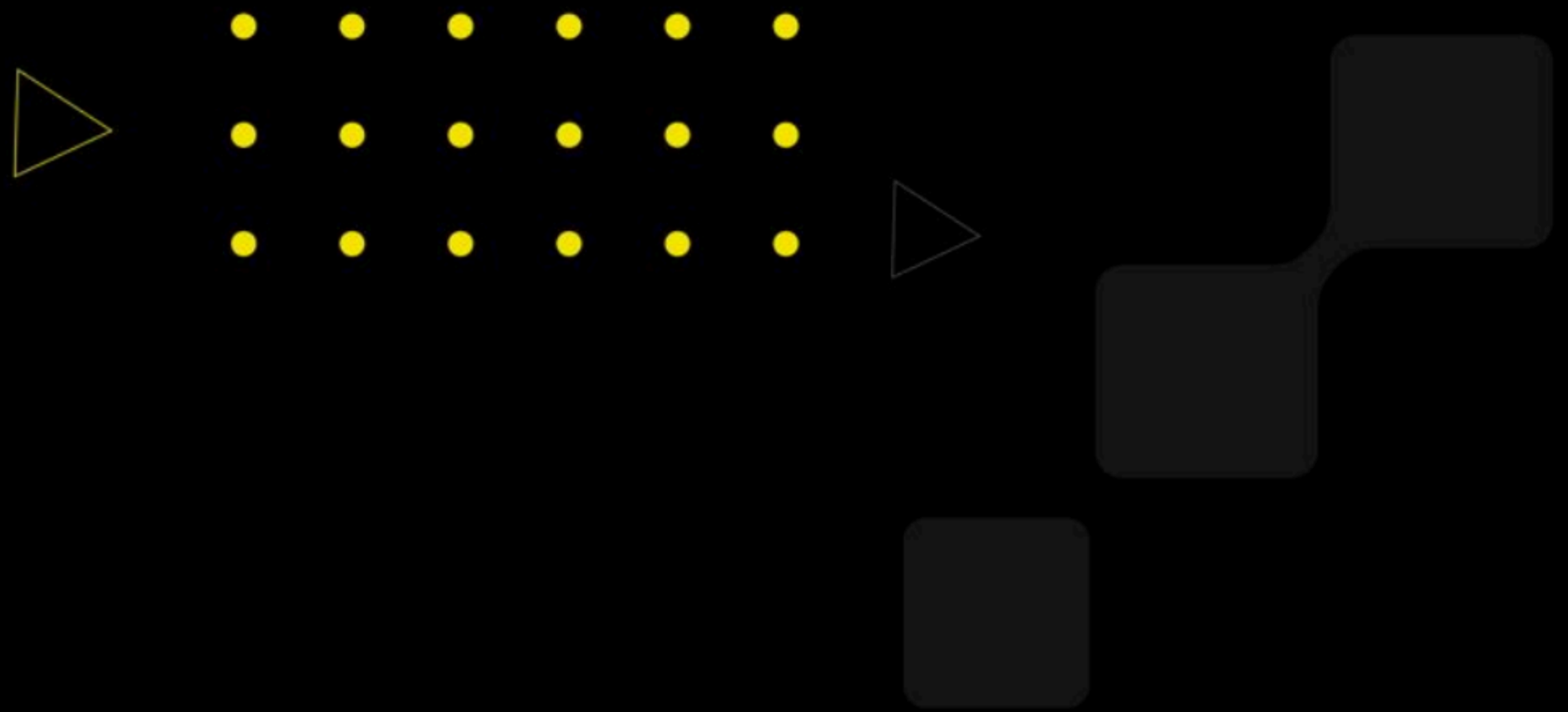


Two separate dev teams

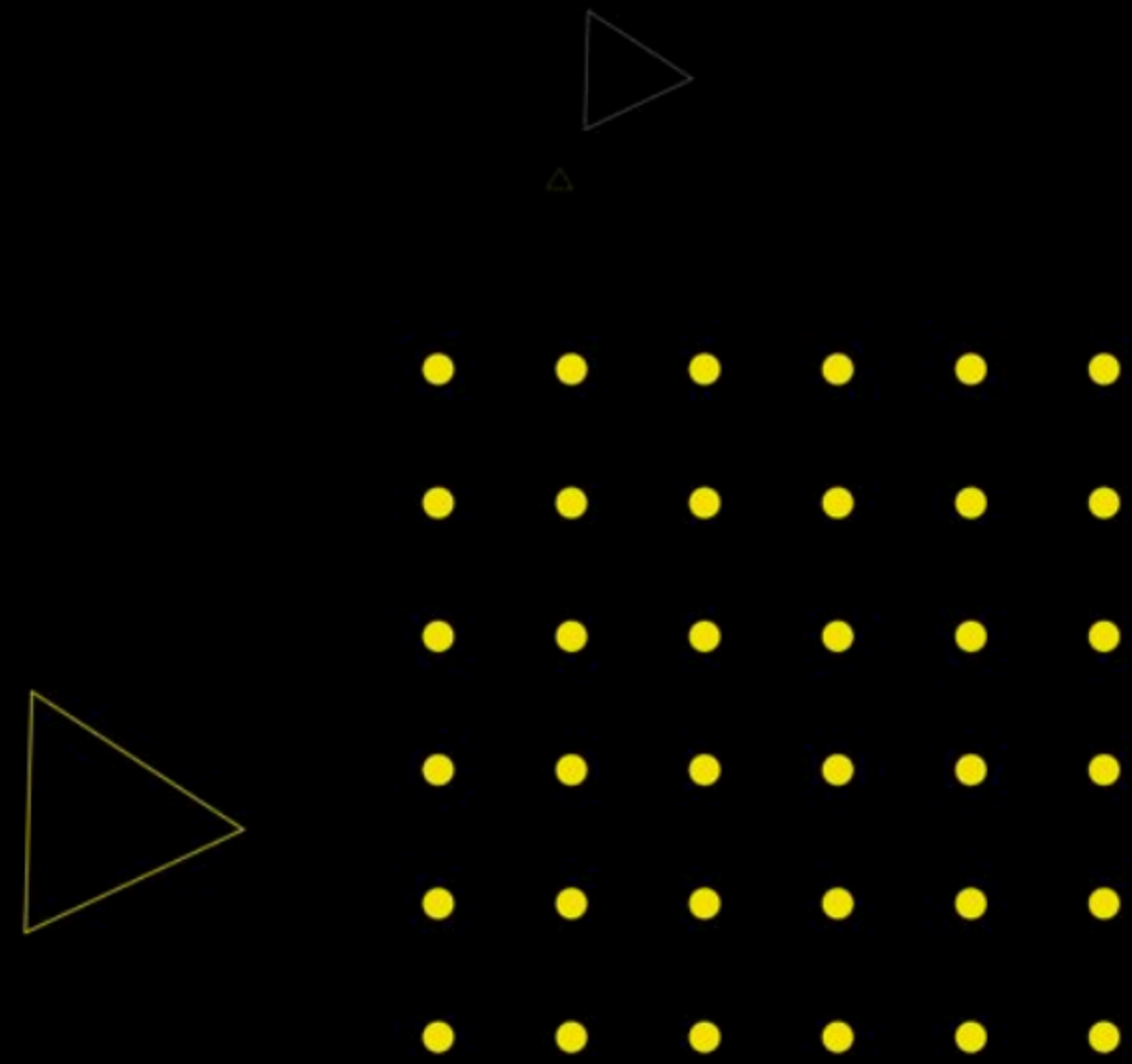


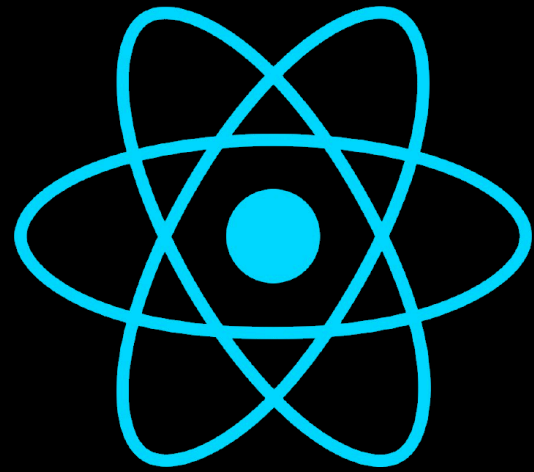
...and separate bugs, release cycles, deployments





It costs a lot

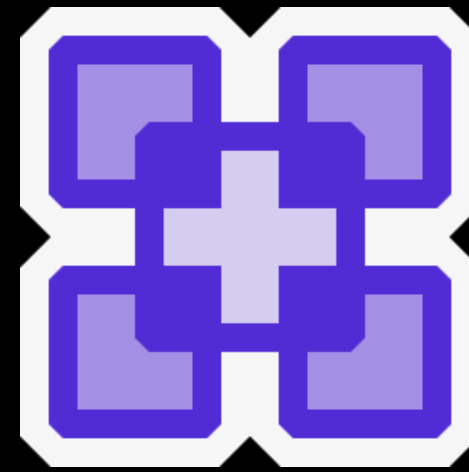




React Native



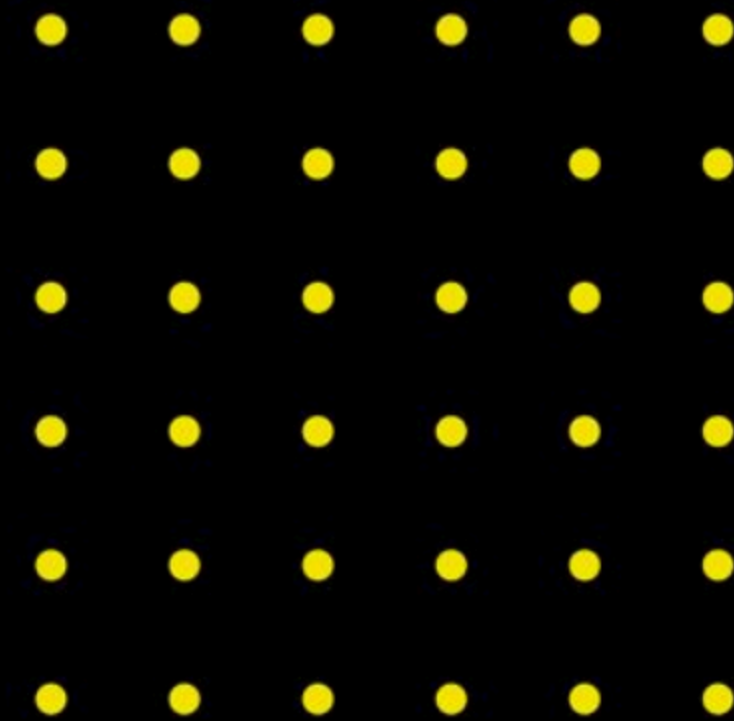
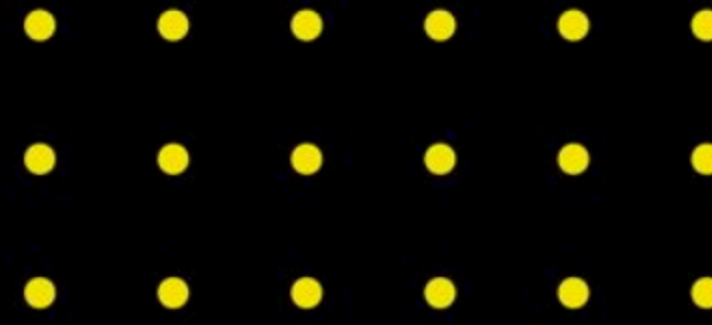
**Compose
Multiplatform**



.NET MAUI



Capacitor





Flutter

Design languages

Dart

Cupertino

Material

Others

Framework

Dart

Widgets

Rendering

Animation

Painting

Gestures

Foundation

Engine

Dart/C++

Compositor

Embedding

I/O

Accessibility

Browser

HTML/CSS

JavaScript

WebGL

WebAssembly

Dart

- Statically typed
- Object-oriented
- Garbage-collected
- Asynchrony support
- 100% sound null safe
- Open source, developed by Google

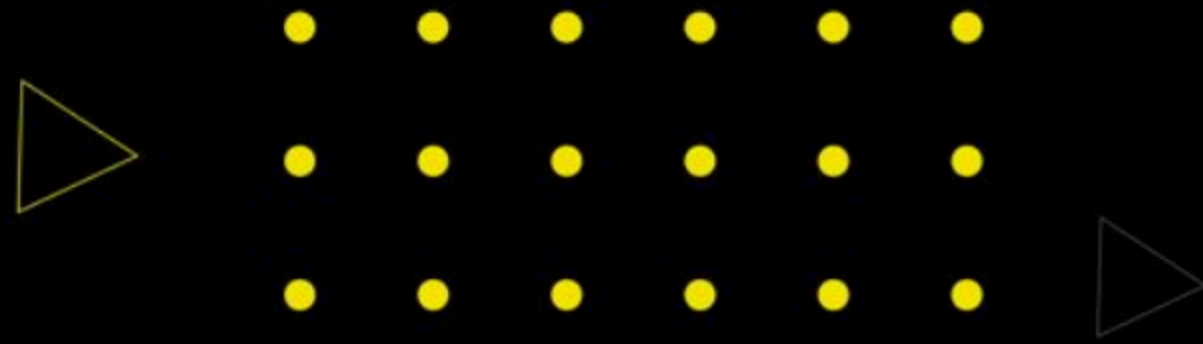
```
int fibonacci(int n) {  
    if (n == 0 || n == 1) return n;  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

```
var result = fibonacci(20);
```

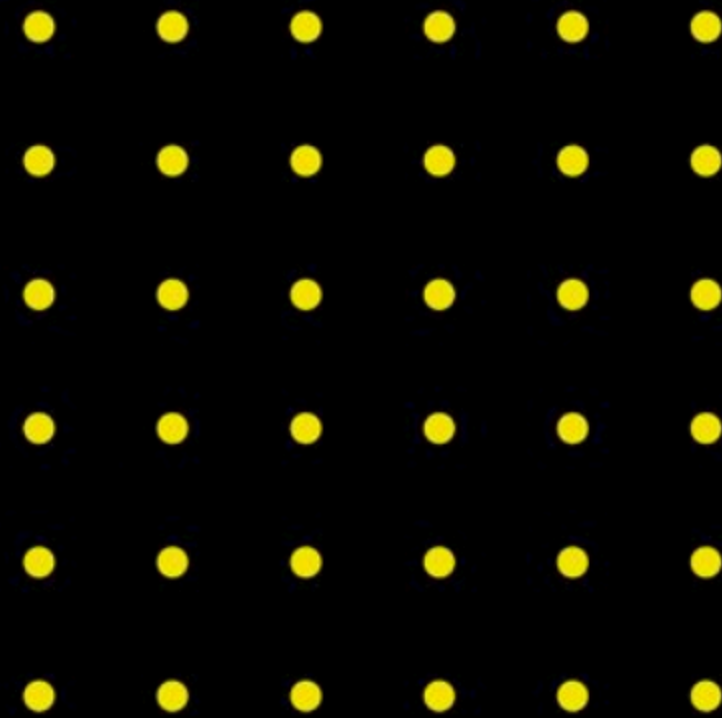
```
var (a, b) = ('left', 'right');  
(b, a) = (a, b); // Swap.  
print('$a $b'); // Prints "right left".
```

Why Dart? - TLDR

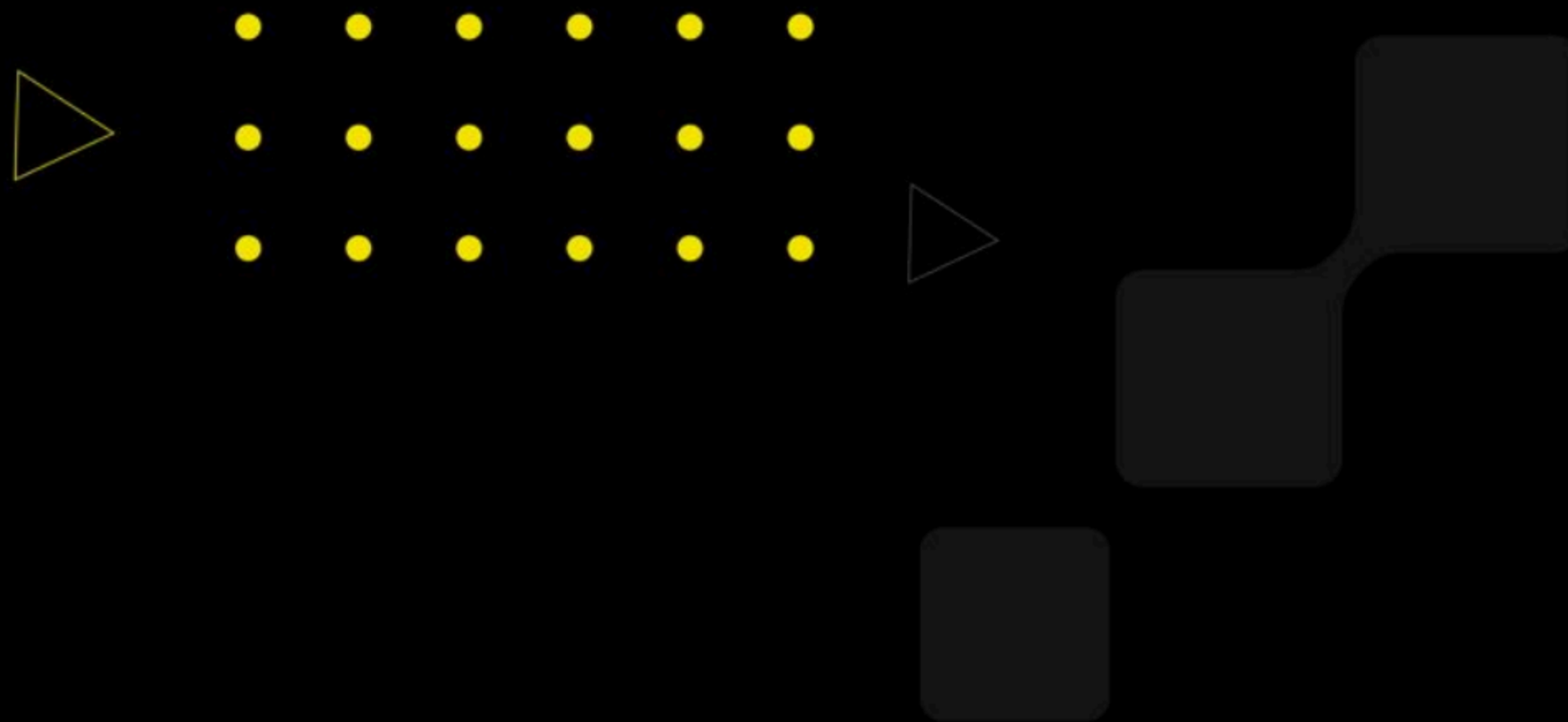
- It looks like JavaScript
- It is fast enough (we have 16 or even 8ms to render a frame)
- It's garbage-collected
- It can perform tree shaking to reduce code size
- It's tightly connected to Flutter use cases
- It's targeted to run on mobile/desktop/web.



Flutter == multi-platform UI toolkit

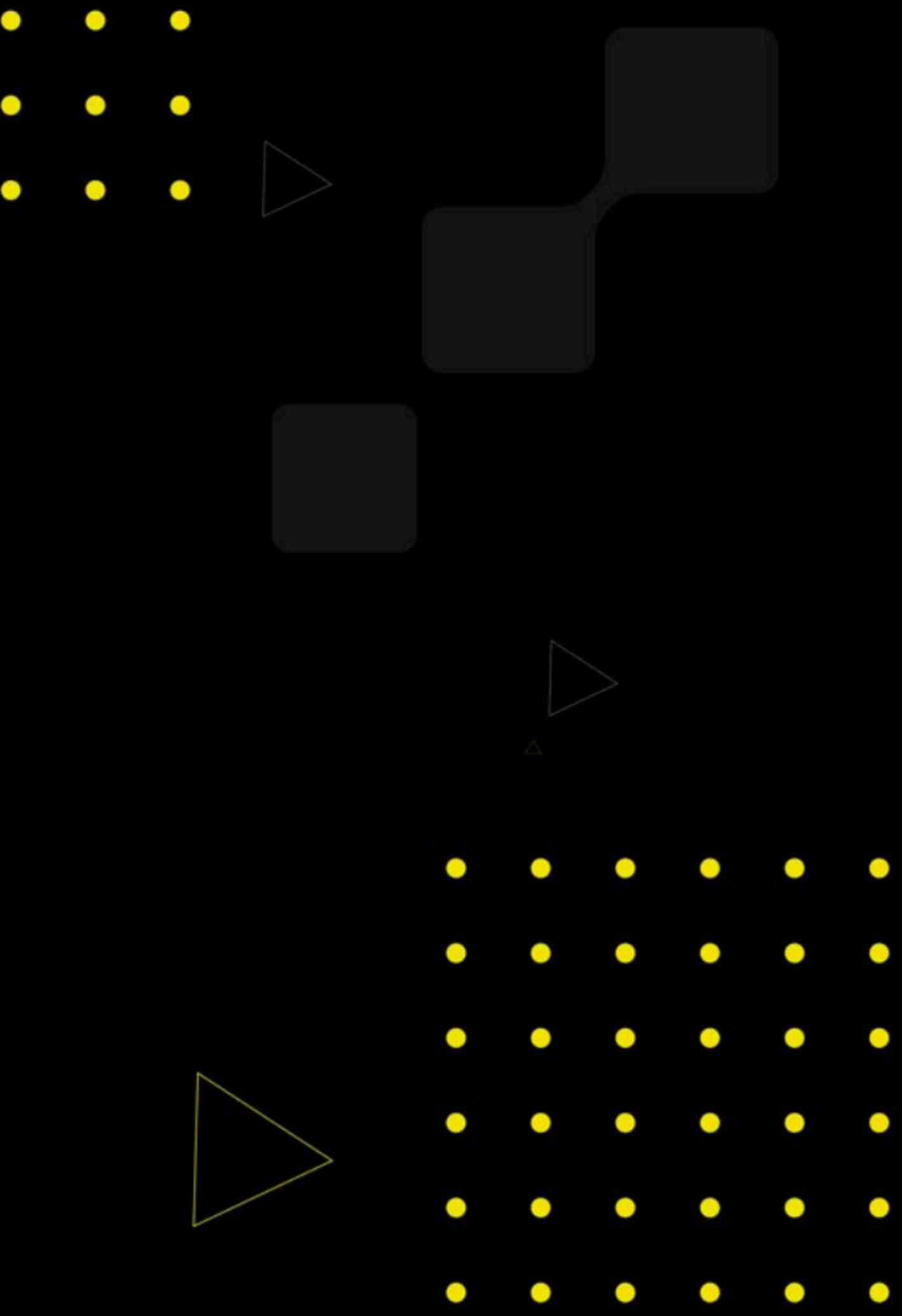


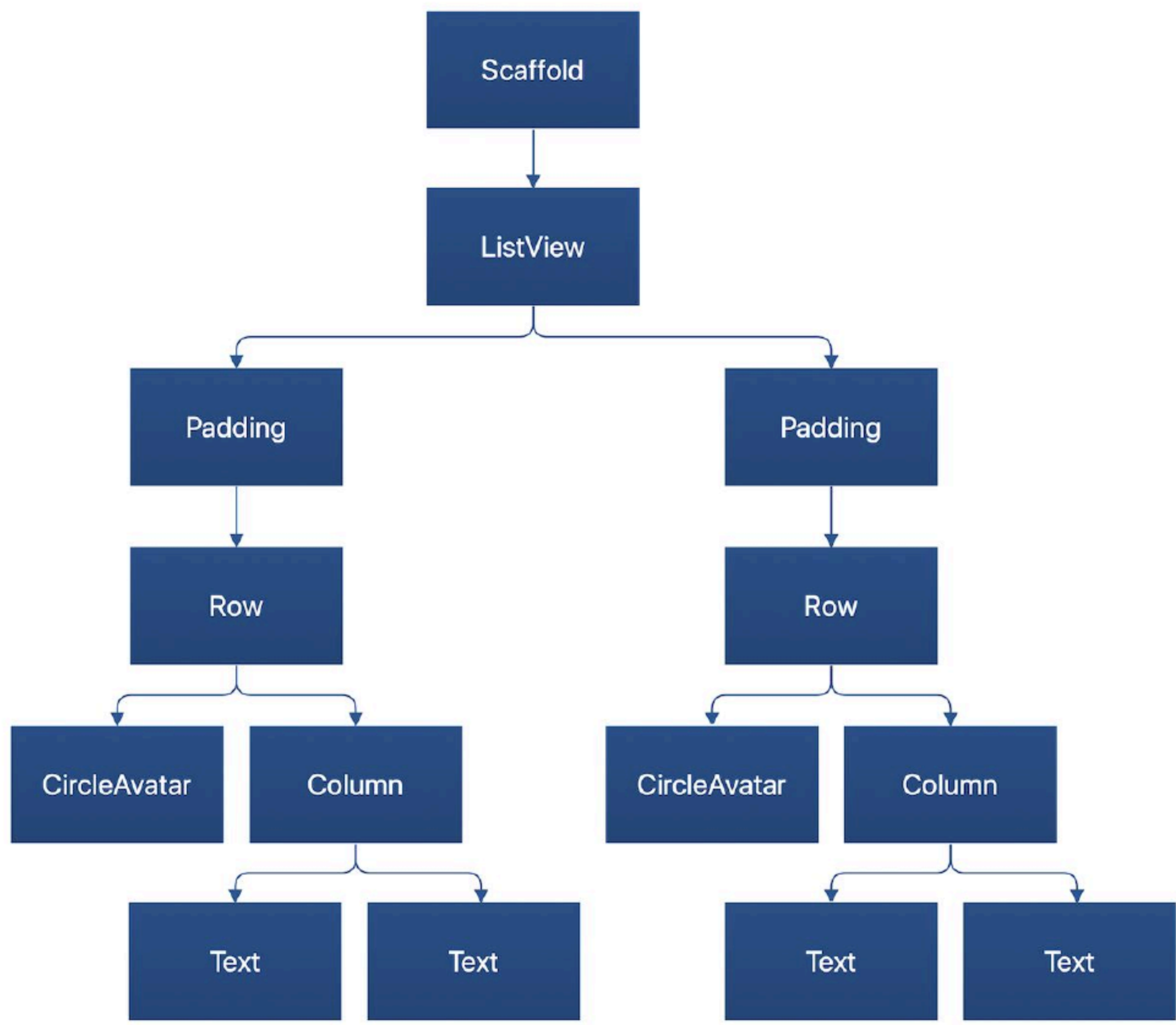
**(Almost) everything is a
widget**

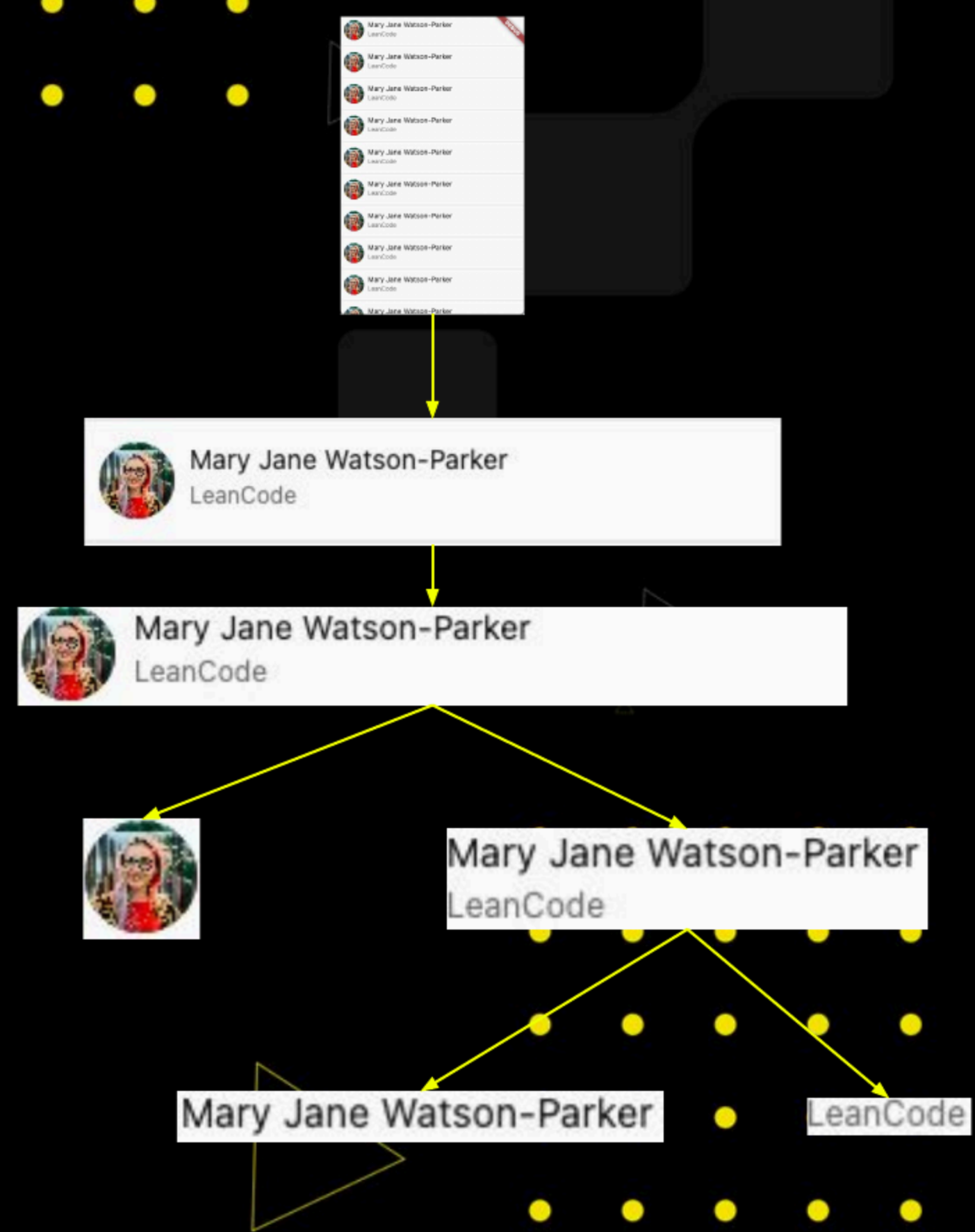
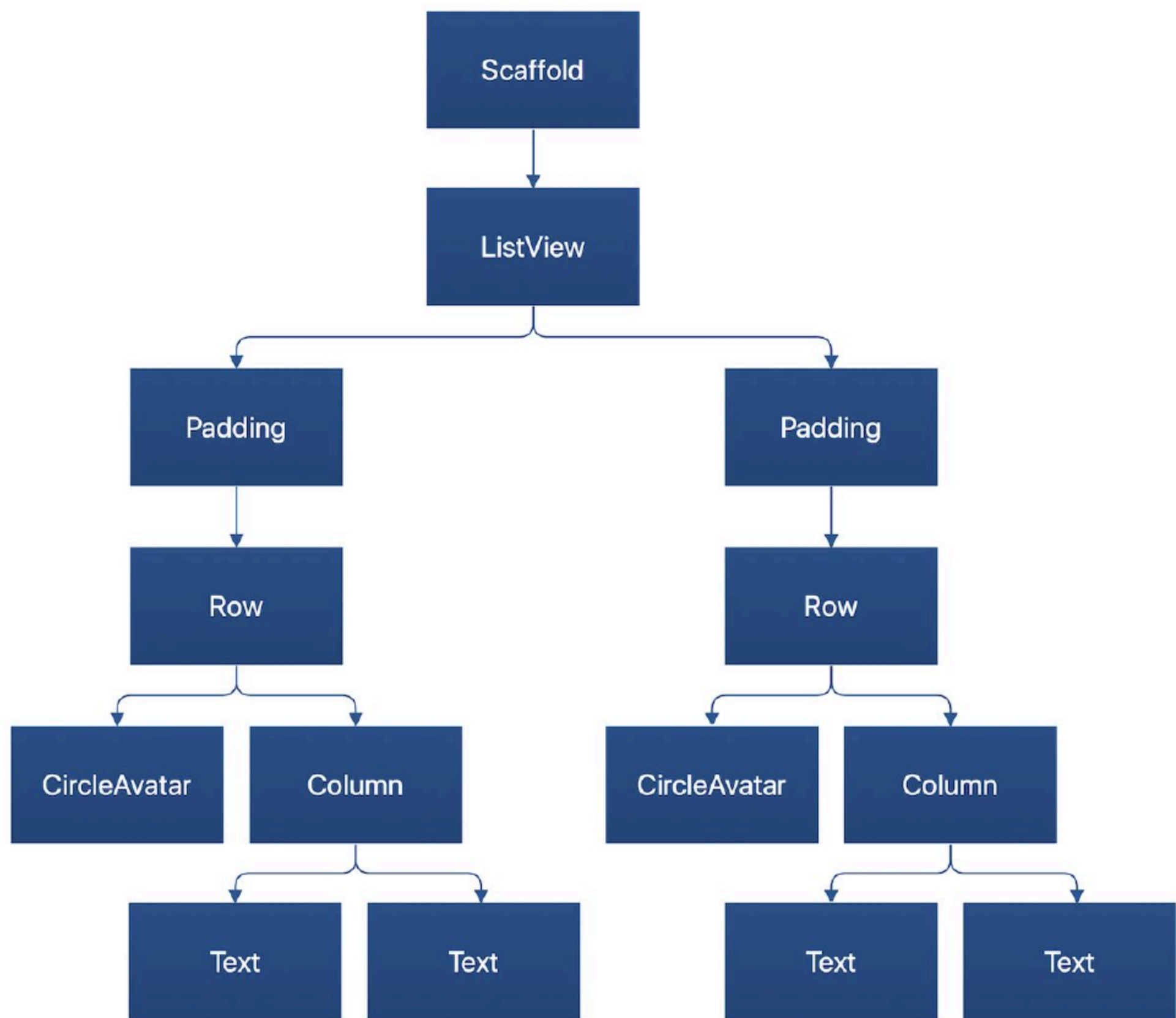


Widget in a widget









Flutter Desktop

macOS + Windows + Linux

Flutter Web

Now using **WebAssembly**

Flutter Embedded

What is Declarative UI?



Imperative UI

Windows Forms / Android / iOS /
GTK

Add callback which on change
does:

- Set color
- Remove child
- Add child
- Set position

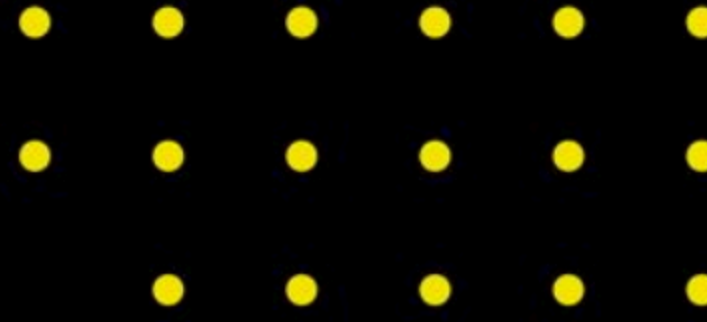


Declarative UI

React / Flutter / Jetpack
Compose / SwiftUI

For this state return a View with red background color and children consisting of a text message and a button.

UI = f(state)



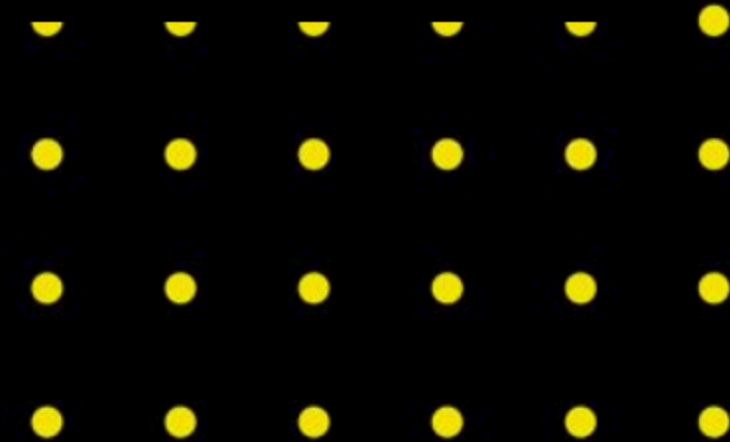
```
final title = Text();
title.data = 'Please tap the button to finish';

final button = Button();
button.text = 'Finish';
button.onPressed = () {
  print('Button pressed!');
};

view.backgroundColor = Colors.white;
view.children = [];
view.children.add(title);
view.children.add(button);
```



```
return View(
  children: [
    Text('Please tap the button to finish'),
    Button(
      text: 'Finish',
      onPressed: () {
        print('Button pressed!');
      }
    ),
  ],
);
```

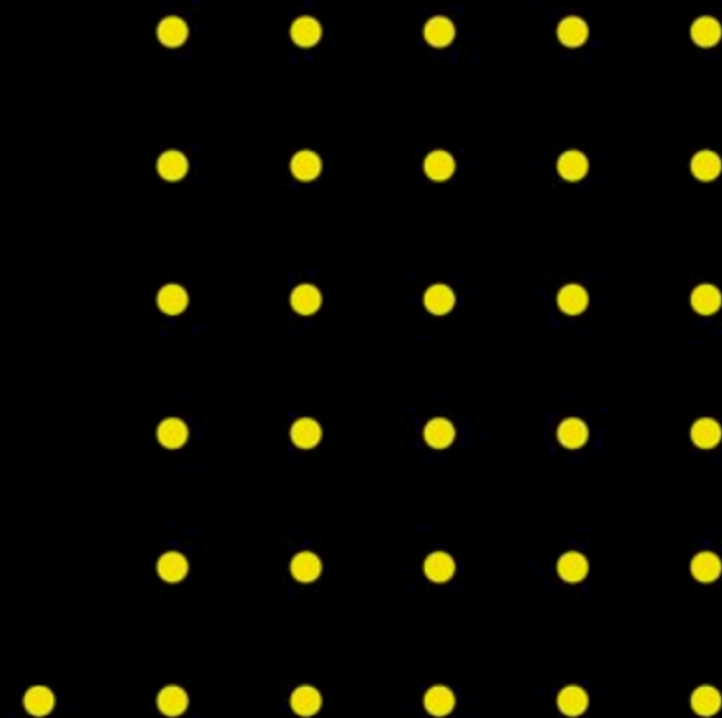


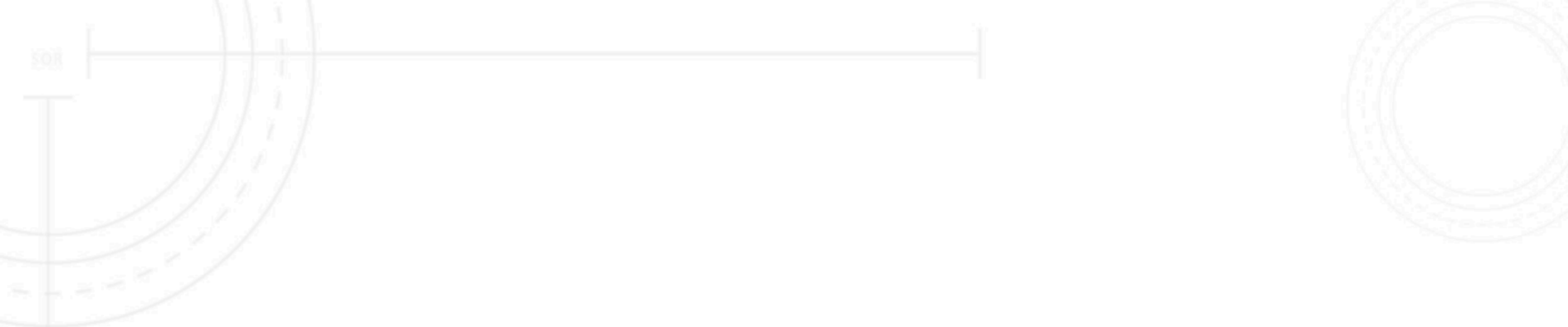
Let's make some Hello World!



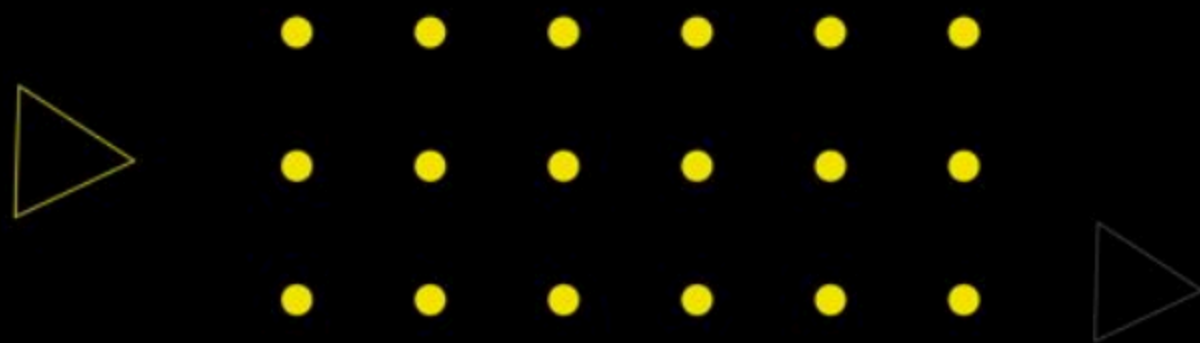
```
import 'package:flutter/material.dart';

void main() {
  runApp(
    const Center(
      child: Text(
        'Hello world!',
        textDirection: TextDirection.ltr,
      ),
    ),
  );
}
```





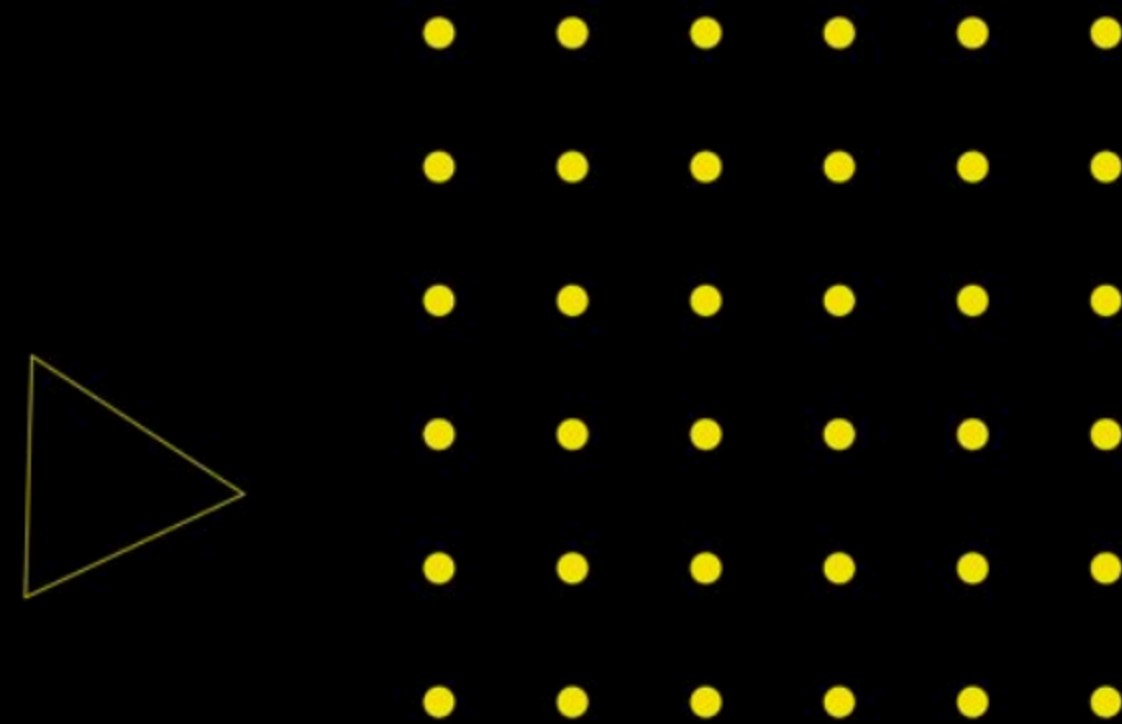
Constraints go down. Sizes go up. Parent sets position.



```
BoxConstraints({double minWidth, double maxWidth, double minHeight, double maxHeight})
```

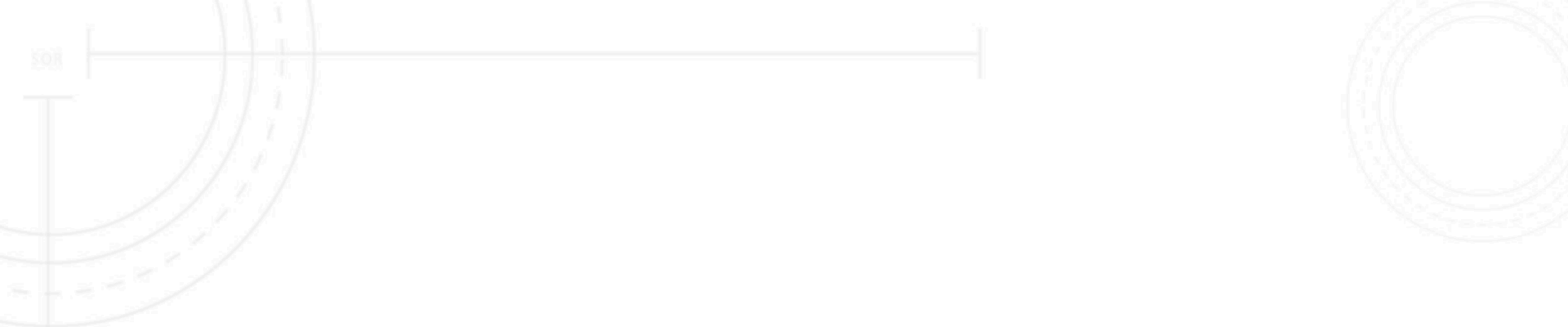
Creates box constraints with the given constraints.

const



Layout algorithm

1. Widget gets constraints from its parent
2. For every child, it requests its size within given constraints (could be different from the first constraints)
3. Knowing children sizes, now the widget positions every of them
4. Knowing children sizes and positions, now the widget can pass its own size to its parent



Show me the code

Lab 1 starts in 15 minutes

- Room **318**: Group 1 at **10:15**, Group 2 at **12:15**
- Bring your laptop if you have one
- Setup guide: github.com/leancodepl/flutter-at-mini
- Solutions by email to your group's teacher: **lab1_kowalski_jan.zip**
- Next week (13.10): Layouts 2 and Lab 2: Layouts 1

Wiktor Zajac · wiktor.zajac@leancode.pl